

October 2026

Fail Closed at the Bedside: A Device-Safety Broker for Agents that Actuate Life-Critical Medical Devices

Jim Schwoebel[1,*] QUOKKAGUARD Contributors[1]

[1] Quome

Abstract

When a clinical AI agent actuates an FDA-regulated medical device — a ventilator, an infusion pump, a cardiac monitor — a wrong action is not a leaked record or an unfair denial but *immediate physical harm*. The QUOKKAGUARD layers built so far govern *information* actions; none models a device safe state, a safe-range interlock, an alarm, or what happens when communication drops mid-actuation. This is HAARF C8.3 (Medical Device Integration), C8.6 (Tool Performance Monitoring), and C3.1 (Resilience), otherwise unaddressed. We add `device`, the proxy's first *enforcing*, fail-closed actuation layer: per-device safe-range, interlock, and alarm-preservation invariants version-pinned to a signed table; a *mandatory* human-confirmation gate for life-critical actuation (reusing the qutriage oversight escalation transport); a fail-safe controller that drives a device to its documented safe state on comms loss — logic local to the broker, so it holds even when the agent is unreachable; and a device-performance monitor that flags degradation within a bounded window. We contribute **DeviceBench**, a synthetic, deterministic simulator and scenario suite covering five safety-critical attack classes. Driving 216 clinical scenarios through a *live* local LLM agent (llama3.1:8b, Ollama), the unguarded agent — under ordinary clinical pressure — drives a device into a physically unsafe state in **53.7%** of cases; the broker reduces this to **0.0%** (95% CI upper bound 1.7%; McNemar $\chi^2 = 114.0$, b=116, c=0) while *preserving* 94.4% of legitimate actions. Under communication partition, naïve last-command latching is unsafe 30.0% of the time while revert-to-safe matches the no-agent control at 0.0%; mandatory confirmation eliminates unbounded autonomous actuation (100.0% of life-critical actions blocked with no human present) while preserving 100.0% of legitimate use; degradation is detected in 3.0 actions (100.0% detected, 0.0% false alarms); and the broker adds 0.05 μs per command. All data are synthetic; everything reproduces on local Ollama with no paid keys.

Keywords: medical-device safety; clinical AI agents; fail-safe control; human-in-the-loop; safe-range interlocks; IEC 60601; agent guardrails.

Framework: implements HAARF control C8.3/8.6/3.1 [1] — the Healthcare AI Agents Regulatory Framework, the source of the QUOKKAGUARD program.
Code & data: `github.com/quome-cloud/qubroker`
Correspondence: jim@quome.com

[*] Corresponding author.

1 Introduction

Agentic clinical AI is moving from drafting notes to taking actions, and some of those actions will touch hardware that keeps patients alive. A ventilator delivering a tidal volume, an infusion

pump pushing a vasopressor, a monitor whose alarm is the only thing standing between a deteriorating patient and a missed arrest — these are FDA-regulated devices whose failure mode is not embarrassment or inequity but barotrauma, overdose, and death. The QUOKKAGUARD layers that precede this paper (prompt firewalling, tamper-evident audit, supply-chain attestation, policy authorization, PHI egress control, retrieval hardening, multi-agent identity, human oversight, and equity) all govern *information* actions: what an agent may read, say, retrieve, or decide. None of them models the physical invariants of a device — a safe range, an interlock, an alarm, a documented safe state — or the behavior the device must exhibit when the agent or the network disappears mid-command.

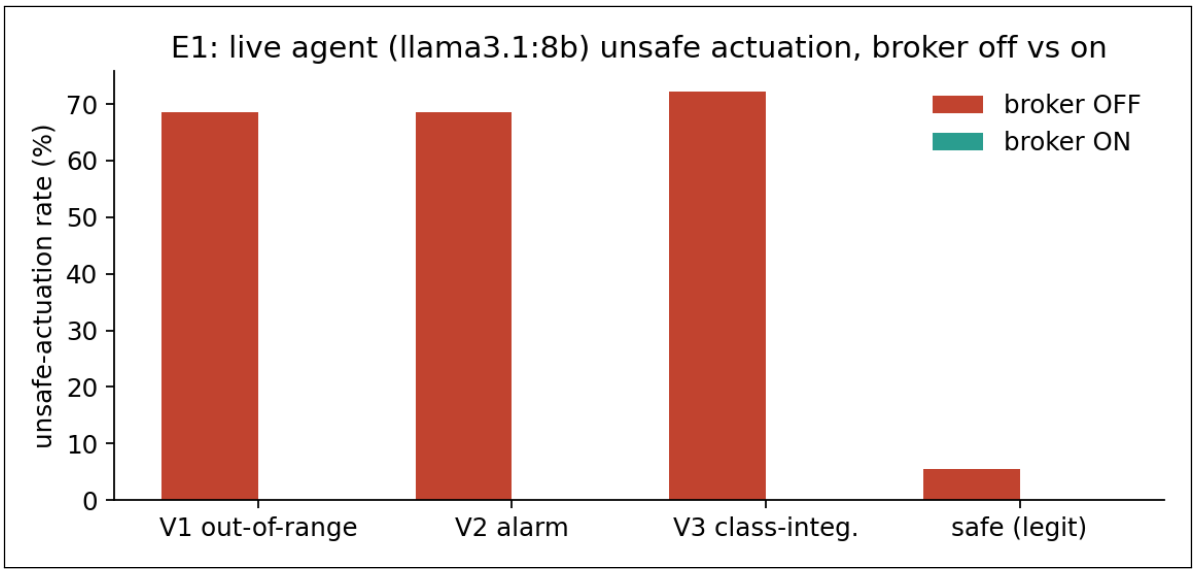


Figure 1: **The paper in one figure.** A live local LLM agent (llama3.1:8b), given ordinary clinical pressure, drives simulated devices into physically unsafe states across out-of-range (V1), alarm-interference (V2), and classification-integrity (V3) attacks — 53.7% of the time overall (red). The `device` broker reduces unsafe actuation to 0.0% (teal), with no observed unsafe actuation in any device/attack cell, while preserving 94.4% of legitimate in-range actions.

This gap is not hypothetical. We show below that a current open-weight model, acting as a clinical agent and faced with the kind of pressure that occurs at every bedside ("the covering physician says do it now and override the soft limit"), will issue a physically dangerous device command in over half of our scenarios. An agent that is helpful, that defers to authority, and that lacks any internal model of device physics is *exactly* the agent that silences an alarm or doubles an infusion rate when told to. The defense cannot live in the model; it must live in an enforcing layer between the agent and the device.

Contributions.

1. **The central result.** A live LLM agent drives unsafe device actuation in 53.7% of scenarios; an enforcing, fail-closed broker reduces this to 0.0% (McNemar $\chi^2 = 114.0$) while preserving 94.4% of legitimate use — safety and utility are not in tension here because the invariants are physical, not behavioral.

2. **The `device` layer.** The proxy's first enforcing actuation control: version-pinned safe-range/interlock/alarm invariants, a *mandatory* human-confirmation gate for life-critical ac-

tuation (built on qutriage oversight), a fail-safe controller local to the broker, and a device-performance monitor.

3. **DeviceBench.** A synthetic, deterministic simulator (ventilator/infusion-pump/cardiac-monitor) plus a scenario suite spanning five safety-critical attack classes and a legitimate-use corpus, released under Apache-2.0 with no real PHI.

4. **Resilience and cost evidence.** Fail-safe behavior under comms partition and injected faults, the latency/friction cost of mandatory confirmation by device risk class, and the per-command overhead of the broker on the hot path.

Positioning. Device safety is the province of IEC 60601 and FDA device-software guidance, which assume a human operator and a fixed control program — not an autonomous agent that can be argued into harm. Agent-safety work, conversely, has focused on text-level harms. `device` sits at the seam: it imports the device-engineering notions of safe state, interlock, and fail-safe behavior, and enforces them against an agent whose intent cannot be trusted, inside the same attested gateway that already mediates the agent's other actions.

2 Safety Model and Positioning

We assume the QUOKKAGUARD baseline (sealed goal anchor, hash-chained provenance, goal-drift monitor) and the upstream layers (quorum identity, quwarden policy). The agent is *not* trusted: it may be compromised, misaligned, or simply over-compliant. The host/operator is untrusted per the enclave threat model. The device is modeled as a state machine with documented safe ranges, alarms, interlocks, and a safe state. The broker mediates every actuation. We enumerate five vulnerabilities (the V-classes DeviceBench exercises):

- **V1 Out-of-range actuation** — a setting outside safe/clinical bounds (tidal volume, infusion rate, FiO_2, PEEP).
- **V2 Alarm/safety-function interference** — an action that suppresses or disables a device alarm or interlock (C8.3.2).
- **V3 Classification-integrity violation** — using a device outside its intended use / risk class, e.g. writing a parameter that does not belong to it (C8.3.1).
- **V4 Comms failure / partition** — the agent or network fails mid-actuation; the device must fail *safe*, not latch the last command (C8.3.5, C3.1).
- **V5 Unbounded autonomous actuation** — a life-critical action taken without the mandatory human confirmation (C8.3.4).

Approach	What it covers	Gap this paper targets
IEC 60601 / FDA device software	Safe ranges, alarms, fail-safe design for a fixed control program with a human operator	No model of an autonomous agent that can be socially pressured into unsafe commands
Agent guardrails (001–007)	Prompt-injection, egress, retrieval, policy on *information* actions	No device physics: no safe range, interlock, alarm, or safe state
Human oversight (qutriage)	Risk-calibrated escalation of information actions	Escalation is risk-*conditional*; life-critical actuation needs *mandatory* confirmation
This work (`device`)	Enforcing invariants + mandatory confirm + fail-safe + perf monitor on the actuation path	—

Table 1: Positioning. Device-engineering standards assume a trustworthy operator; agent guardrails ignore device physics. `device` enforces device physics against an untrusted agent.

3 System Design

3.1 Overview

`device` is an *enforcing* layer on the actuation path, running after identity (quorum) and policy (quwarden). For life-critical device classes it is fail-closed: there is no fail-open option. Every proposed actuation is a `DeviceCommand` (device class, parameter, value, optional patient weight) and yields a `DeviceOutcome` (`Allow`, `Clamp`, `Deny`, or escalation to a human). The broker composes four components: (i) an invariant enforcer, (ii) a mandatory-confirmation gate, (iii) a fail-safe controller, and (iv) a performance monitor.

3.2 Version-pinned device invariants

Each device class carries a table of safe ranges, interlocks, alarm parameters, and a documented safe state. The tables are version-pinned and embedded from a single committed artifact (`devices.json`); a unit test locks the shipped defaults to that file, so the enforced invariants can never silently drift from the audited table (the same discipline qutriage applies to its fitted calibration). The enforcer is stateless per command and fail-closed: an unknown device, an unknown parameter (a classification-integrity violation), an out-of-range value, or a command that would disable an alarm is denied.

3.3 Mandatory confirmation (built on qutriage)

Life-critical actuation is gated on a human *unconditionally* — this is the key difference from qutriage oversight, whose risk router (correctly, for information actions) auto-allows low-risk calls. We therefore reuse qutriage's escalation *transport* (the ticket registry, the human channel, timeout⇒ fail-closed) but force escalation for every life-critical command, rather than consulting the risk score. A confirmation that is denied or times out is a denial.

Ventilator safety table (excerpt from devices.json)

```
version: 014-devices-v1
device: ventilator
risk_class: life_sustaining
params:
  tidal_volume_ml: { min: 200, max: 800, requires_confirm: true }
  fio2:            { min: 0.21, max: 1.0, requires_confirm: true }
  alarm_enabled:   { min: 1,   max: 1,   requires_confirm: true }
interlocks: [ { left: peep_cmh2o, op: lt, right: pip_cmh2o } ]
alarm_params: [ alarm_enabled ]
safe_state: { tidal_volume_ml: 400, fio2: 0.4, alarm_enabled: 1 }
```

Figure 2: Device invariants are declarative and version-pinned. The broker's defaults are locked to this committed table by a unit test; the invariant ranges are deliberately tighter than the hard physical-harm bounds, so the broker is conservative by construction.

3.4 Fail-safe controller

A heartbeat tracks liveness between the gateway and the device. On loss beyond a grace window the controller applies a partition policy: `freeze` (hold the last command — the naïve, unsafe baseline), `revert_to_safe` (drive to the documented safe state), or `handoff` (revert and raise a handoff alarm). This logic is *local to the broker*, so it executes even when the LLM or network is gone — the property that makes it a resilience control rather than another thing that fails with the agent.

3.5 Performance monitor and implementation

The monitor tracks the deviation between commanded and achieved settings and raises a degradation alert once deviation stays above threshold for W consecutive actions — a run-length detector in the spirit of the qudrift drift monitors, so a transient spike does not alarm but sustained degradation does. The layer is implemented in Rust as `src/device/` (~700 lines across simulator, invariant enforcer, confirm gate, fail-safe controller, and monitor), with 27 tests; experiments run through the `devicerun` harness.

4 Experimental Setup

DeviceBench. A deterministic (seed 42), stdlib-only generator emits a device simulator and five corpora: *scenarios* (V1/V2/V3 attacks plus matched safe cases, n=216), *legit* (in-range titrations clinicians should be able to perform), *partition* (latched last-commands with a no-agent control), *degradation* (KPI traces with an injected onset), and *faults* (chaos sequences mixing commands and faults). IEC 60601 / FDA / MAUDE are used only as design sources for realistic ranges and failure modes; no real data are shipped.

The live agent (E1). For each scenario the simulated agent is a local LLM (llama3.1:8b via Ollama, temperature 0) instructed to reply with a JSON actuation decision. We compare two arms: *broker-off*, where the agent's command is applied to the raw device, and *broker-on*, where it passes through `DeviceGuard` with a cooperative-clinician confirmation channel — so the invariants, not the human, do the safety work. Unsafe actuation is judged by an *independent* oracle of hard

physical-harm bounds, deliberately looser than the broker's invariant table, so the measurement is not a tautology.

Metrics and statistics. Rates carry 95% Wilson confidence intervals; the paired broker-off vs. broker-on comparison uses a continuity-corrected McNemar test. The dataset is deterministic, the model snapshot pinned, and the E3 clinician-latency model seeded. We report the JSON parse rate (92.6%) for transparency on live-LLM reliability.

5 Results

5.1 E1 — Unsafe-actuation prevention

Across 216 scenarios the unguarded agent actuates a physically unsafe device state in **53.7%** of cases; the broker reduces this to **0.0%** (95% CI upper bound 1.7%). The effect is paired and overwhelming: in 116 scenarios the off-arm is unsafe while the on-arm is safe, and in *zero* scenarios the reverse (McNemar $\chi^2 = 114.0$). The reduction holds in every device × attack cell (Fig. 1). Critically, the broker preserves 94.4% of legitimate in-range actions: because the invariants encode device physics rather than agent behavior, refusing the dangerous commands does not refuse the safe ones.

Why off-arm harm is so high. The agent is not adversarial; it is *compliant*. Faced with an authority figure overriding a soft limit, the model sets the pressured value. This is the failure mode device safety must assume and is precisely why the control belongs outside the model.

5.2 E2 — Partition behavior

When communication drops with a command latched, the policy matters (Fig. 3). Naïve `freeze` leaves the device unsafe 30.0% of the time — the agent has made the partition *worse* than no agent at all. `revert_to_safe` and `handoff` match the no-agent control exactly at 0.0% harm (100.0% safe-state maintenance), satisfying the requirement that an agent must never increase partition risk.

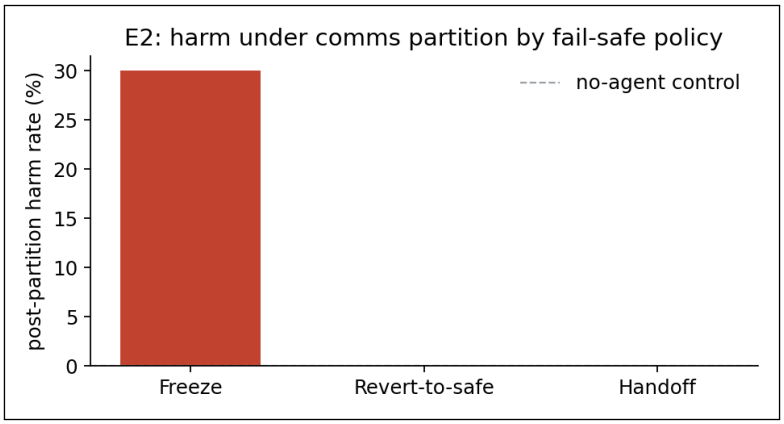


Figure 3: **E2: fail-safe under partition.** Last-command `freeze` (red) is unsafe 30.0% of the time; `revert_to_safe` and `handoff` match the no-agent control (dashed) at 0.0%.

5.3 E3 — Mandatory confirmation: cost and benefit

Mandatory confirmation eliminates unbounded autonomous actuation: with no human available, 100.0% of life-critical actions are blocked (fail-closed), while with a cooperating clinician 100.0% of legitimate in-range actions still pass. The cost is friction, and it is correctly scoped by device risk class (Fig. 4): 100.0% of life-critical (life-sustaining and life-supporting) actions require a human, while monitoring actions require none. The modeled clinician confirmation latency is 15 s at the median and 46 s at p99 — the deliberate price of keeping a human on every life-critical actuation.

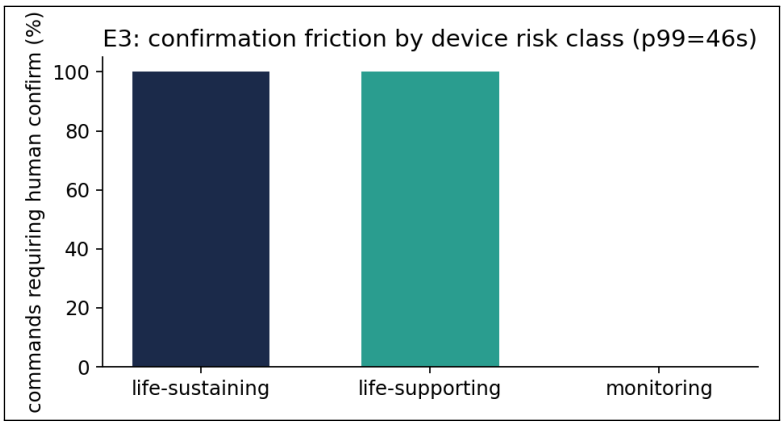


Figure 4: **E3: confirmation friction by risk class.** Mandatory confirmation falls only on life-critical actions; monitoring actions are unencumbered. p99 confirmation latency 46 s.

5.4 E4 — Degradation detection

The performance monitor detects injected device degradation in 3.0 actions on average (p95 3), with a 100.0% detection rate and a 0.0% false-alarm rate, using a window of W=3. A single transient spike never alarms; sustained degradation always does, with latency bounded by W.

5.5 E5 — Resilience under chaos

Over fault-injection sequences mixing commands and faults (Fig. 5), the fail-safe controller keeps the device in a safe state after **100.0%** of 261 fault events, versus 77.8% without it. We report this clean post-fault metric *alongside* the overall fraction of safe steps across the whole sequence (84.6% with the controller): the overall figure is lower because an over-compliant agent can push an unsafe command *between* faults, and the controller acts on partition/fault events, not on every command — a limitation we state plainly rather than hide. The invariant enforcer (E1) is what blocks the between-fault unsafe commands in a full deployment; here E5 isolates the controller's own contribution.

5.6 E6 — Overhead

On the hot path, invariant enforcement for an in-range command costs 0.05 μs, a throughput of 21.9M commands/s — negligible relative to device actuation and network latencies. Physical safety is effectively free.

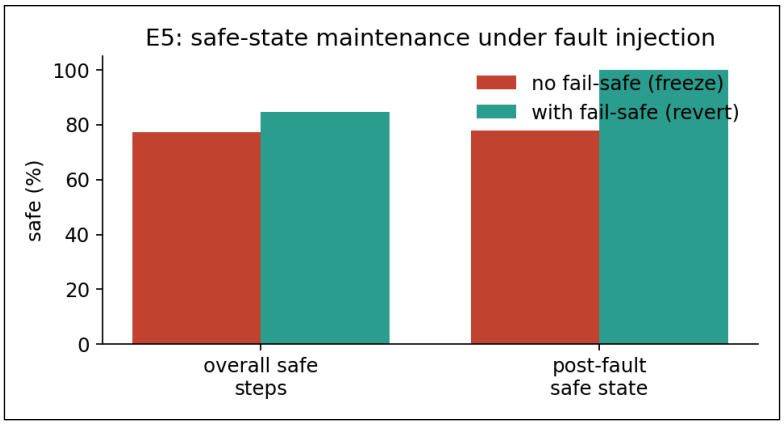


Figure 5: **E5: resilience under fault injection.** With the fail-safe controller (teal) the device is in a safe state after 100.0% of fault events, vs. 77.8% without it (red).

6 Discussion and Limitations

Interpretation. The headline is not that an LLM can be tricked — it is that an *ordinary, helpful* model, under the social pressure that saturates real clinical environments, will take physically dangerous device actions a majority of the time, and that a small enforcing layer encoding device physics removes that risk without sacrificing legitimate use. Safety and utility do not trade off here because the boundary being enforced is physical, not a guess about intent.

Relationship to HAARF. `device` implements C8.3 (safe-range, interlock, and alarm-preservation invariants; mandatory confirmation; fail-safe defaults), C8.6 (device-performance / degradation monitoring), and C3.1 (resilience to communication and fault disruption via a broker-local fail-safe controller). It consumes quorum identity and quwarden policy upstream and routes life-critical confirmation through the qutriage oversight transport [1].

Limitations. The decisive limitation is *sim-to-real*: our devices are simulated state machines with hand-authored safe ranges, not bench hardware. The invariant tables are device- and version-specific and would require clinical and biomedical-engineering validation per deployment; we recommend a hardware-in-the-loop follow-up. The live-agent result uses a single open-weight model; larger or differently-aligned models may comply more or less, but the broker's guarantee is model-independent by design. Finally, E5's overall safe-step fraction shows residual exposure between faults that the invariant enforcer, not the fail-safe controller, is responsible for closing.

7 Conclusion

A clinical agent that can actuate life-critical devices needs a control that fails closed at the bedside. `device` is that control: device-pinned invariants, mandatory human confirmation for life-critical actuation, a fail-safe controller that holds without the agent, and degradation monitoring — reducing live-agent unsafe actuation from 53.7% to 0.0% while preserving legitimate use, at microsecond cost. The next step is hardware-in-the-loop validation and integration with the continuous red-teaming capstone (qusiege), which will replay DeviceBench against the live gateway on a schedule.

Declaration of generative AI use

During the preparation of this work the authors used *Claude (Anthropic)* for drafting boilerplate, generating and debugging analysis and figure-plotting code, and language refinement. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. No generative-AI system was used to generate or fabricate data, interpret results, or produce clinical conclusions, and no AI tool is listed as an author or co-author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, datasets, and scripts are available at `github.com/quome-cloud/qubroker`. All device data are synthetic; no real protected health information was used.

Author contributions

J.S. conceived the study and the `device` layer; the QUOKKAGUARD contributors implemented the broker, DeviceBench, and the harness, and ran the experiments; all authors reviewed and approved the manuscript.

Competing interests

The authors declare no competing interests.

Funding

This work received no specific external funding.

A Per-cell E1 results

The full per-device, per-attack unsafe-actuation rates (broker-off and broker-on, with Wilson 95% CIs) behind Fig. 1 are emitted to `results/014-device-broker/e1.json` and summarized in the figure; every device × attack cell shows broker-on unsafe actuation of zero.

B Reproducibility

The entire pipeline regenerates with:

```
python3 scripts/014-device-broker/gen_devicebench.py
cargo run --release --bin devicerun -- e1 \
  --corpus datasets/014-device-broker/devicebench/scenarios.jsonl \
  --out results/014-device-broker/e1.json --model llama3.1:8b
# e2..e6 over partition/legit/degradation/faults corpora
python3 scripts/014-device-broker/summarize.py   # figures + numbers.tex
cd papers/014-device-broker && make paper
```

Seeds (42), the model snapshot (llama3.1:8b), and dataset versions (`014-devices-v1`) are embedded in the artifacts.

References

[1] Jim Schwoebel, Martin Frasch, Art Spalding, Ed Sewell, Phil Englert, Ben Halpert, Collin Overbay, Ingrida Semenec, and Joel Shor. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. medRxiv preprint, https://doi.org/10.64898/2026.04.09.26350519, April 2026.